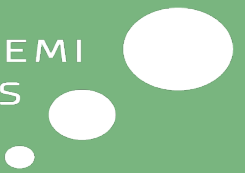
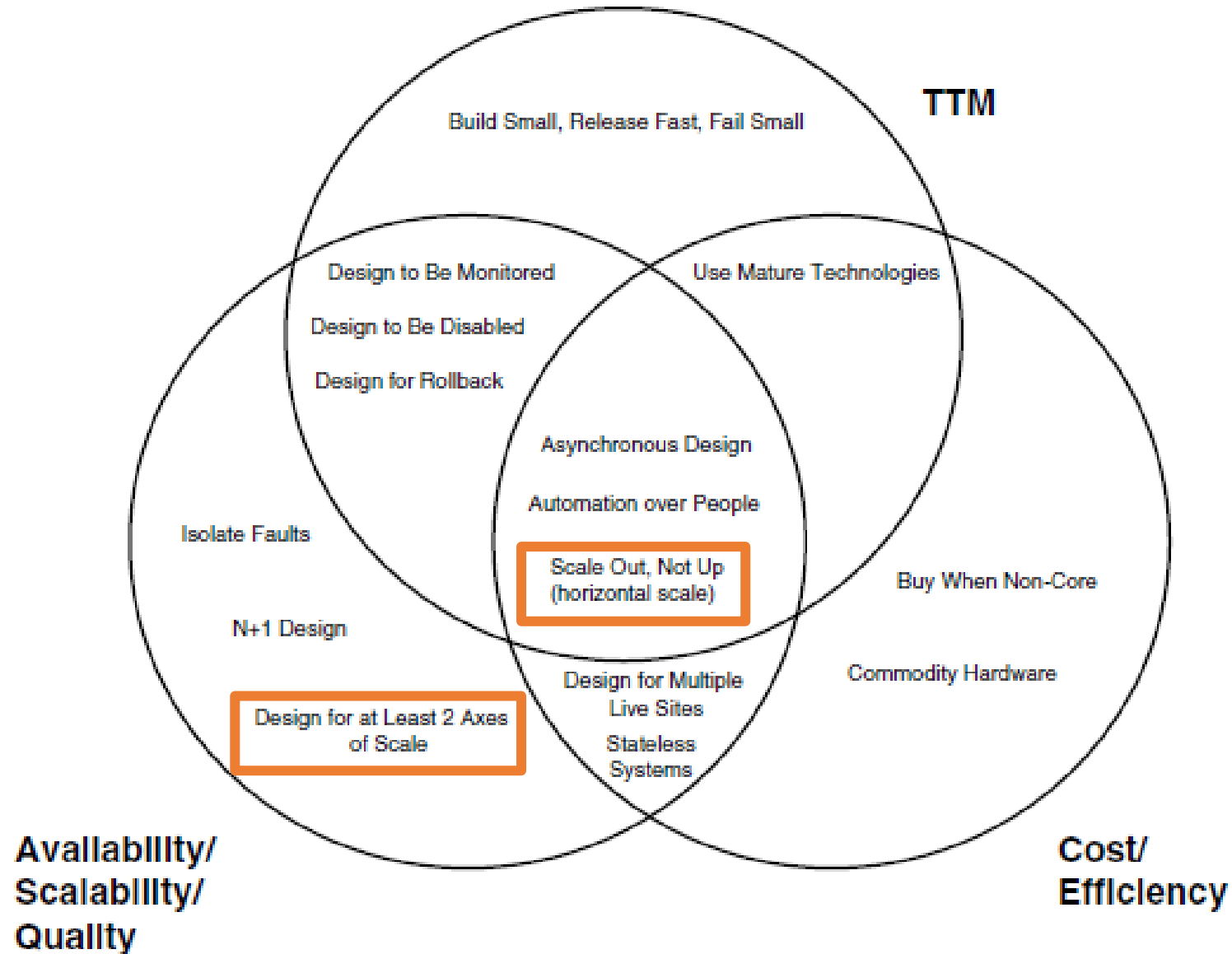


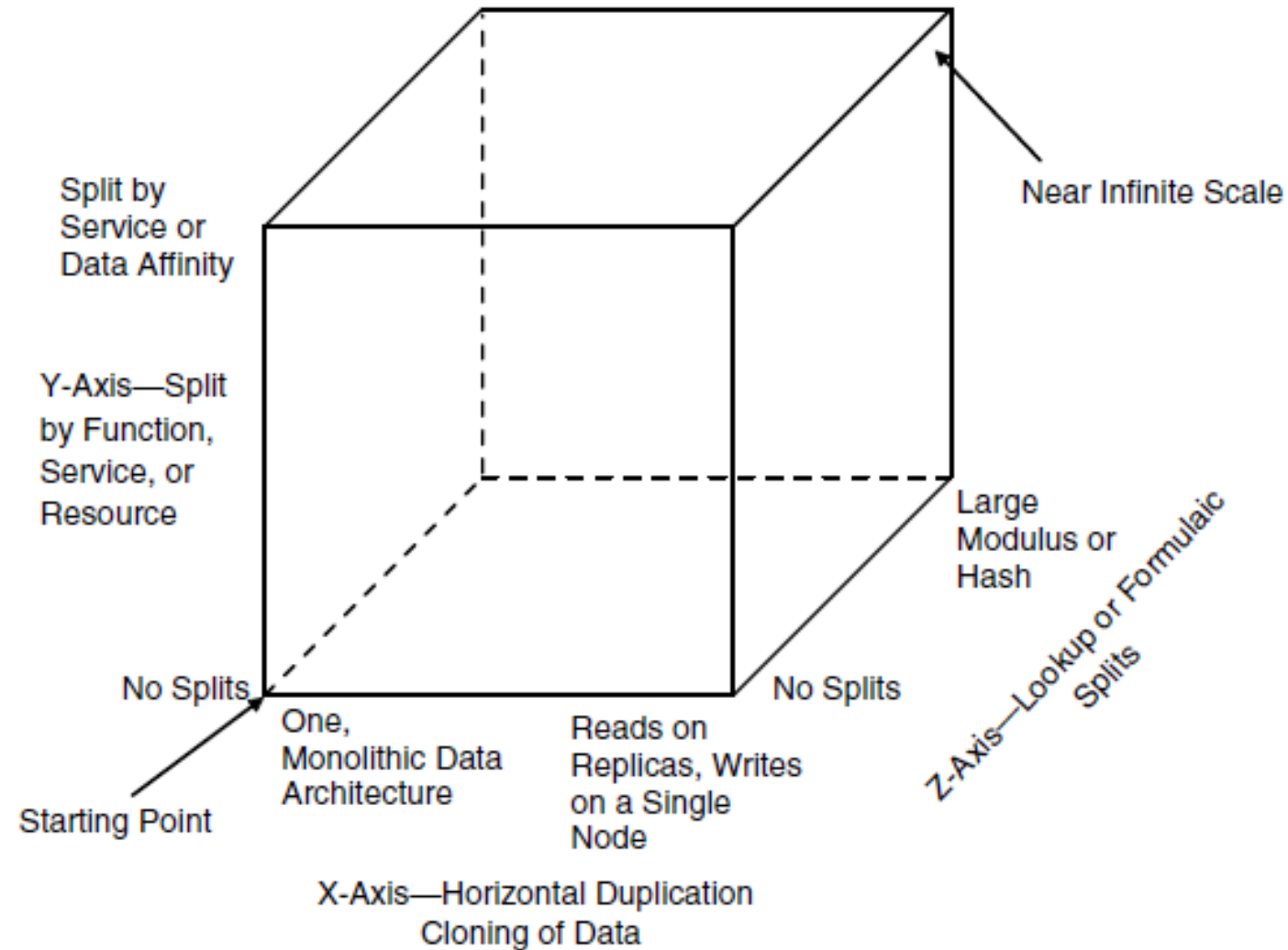
Splitting Databases



Arkitekt Principper i Operations



AKF Database Scale Cube



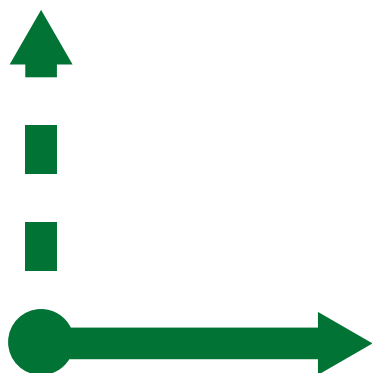
I forbindelse med databaseopdeling, refererer **biased data** til data, der er organiseret eller segmenteret, baseret på specifikke karakteristika, temaer eller relationer.

For eksempel:

- **Y-akse bias:** Data grupperes efter **funktionalitet** eller **ressourcetype** (f.eks. kundedata vs. produktdata), i overensstemmelse med specifikke applikationsoperationer eller temaer.
- **Z-akse bias:** Data opdeles baseret på opslagsværdier eller kriterier som kundegeografi eller en hashfunktion, uden hensyntagen til indholdets tematiske relationer.

I modsætning hertil kloner **unbiased data** (f.eks. i x-akseopdelinger) fuldt ud uden segmentering eller organisation baseret på sådanne karakteristika, hvilket sikrer ensartet duplikering på tværs af alle noder.

Opdeling af database: X-Axis



At opdele en database langs **x-aksen** betyder at skabe **identiske replikaer** af hele databasen for at distribuere læseoperationer og håndtere høje transaktionsbelastninger.

Data klones **uden bias**, hvilket sikrer *eventuel konsistens* på tværs af knuder, med skrivinger rettet til en enkelt "**master**" for at opretholde **ACID-egenskaberne**.

Det er en ligetil tilgang for at skalere **transaktionsrater**, men adresserer ikke **datavækst**.

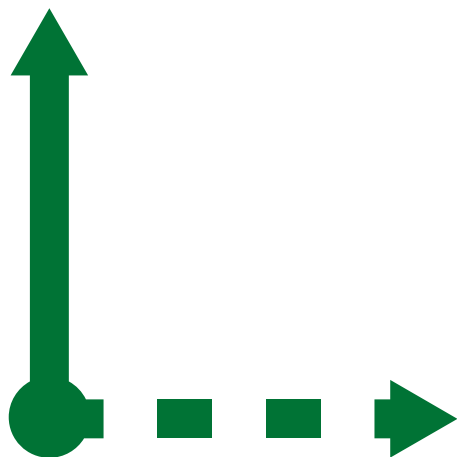
Fordele:

- **Bedre læseskalerbarhed:** Flere replikaer deler læsebelastningen.
- **Øget tilgængelighed:** Andre replikaer tager over ved fejl.
- **Nem opsætning:** Hurtigere end y- eller z-aksedelinger.
- **Ens data på alle noder:** Samme datasæt på alle replikaer.
- **God til læsetunge systemer:** Perfekt ved mange læseforespørgsler.

Ulemper:

- **Begrænset skalerbarhed:** Hele datasættet kopieres til hver replika.
- **Forsinket replikering:** Ændringer opdateres ikke straks.
- **Skrive-flaskehals:** Kun primær database håndterer skrivninger.
- **Højt ressourceforbrug:** Kræver ekstra kapacitet.
- **Ingen fejlisolering:** Fejl påvirker alle replikaer.

Opdeling af database: Y-Axis



At opdele en database langs **y-aksen** betyder at opdele databasen i **funktionelle** eller **tematiske segmenter** baseret på betydning, brug eller ressourcer (f.eks. kundedata, produktdata).

Hvert segment fungerer uafhængigt, hvilket **reducerer kompleksitet** og konflikter og er ofte tilpasset specifikke applikationsfunktioner.

Denne tilgang **hjælper med at styre datavækst** og kompleksitet, men kræver en **betydelig udviklingsindsats**.

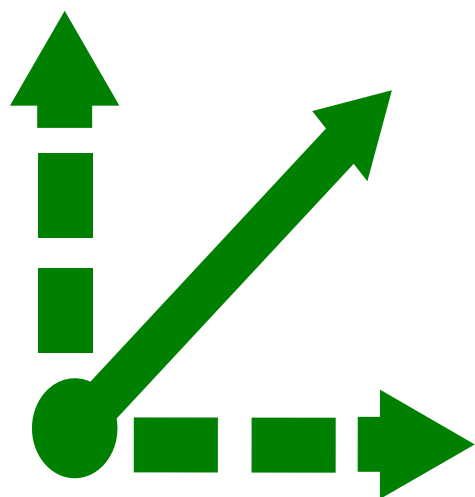
Fordele:

- **Mindre kompleksitet:** Data opdeles i temaer.
- **Bedre fejlisolering:** Ét segments fejl påvirker ikke andre.
- **Skræddersyet optimering:** Segmenter kan optimeres individuelt.
- **Skalerer med kompleksitet:** Belastning fordeles på flere skemaer/databaser.
- **Støtter teamarbejde:** Teams kan arbejde uafhængigt.

Ulemper:

- **Høje omkostninger:** Kræver omfattende arbejde med data og kode.
- **Begrænset skalerbarhed:** Få segmenter løser ikke stor vækst.
- **Øget kompleksitet:** Flere databaser skal driftes.
- **Dataoverlap:** Risiko for duplikering og synkroniseringsproblemer.
- **Svær migration:** Vanskelig overgang fra monolit til segmentering.

Split af database: Z-Axis



At opdele en database langs **z-aksen** betyder opdeling af data baseret på **opslagsværdier** eller **kriterier**, såsom kundeplacering eller et hash af et ressource ID.

Hvert segment (eller shard) indeholder unikke data, der sikrer **afbalanceret belastningsfordeling** og **skalerbarhed**.

Denne tilgang **reducerer konflikter** og **understøtter høj transaktions- og datavækst**, men kræver yderligere mekanismer til at dirigere anmodninger til det relevante shard.

Z-Axis Splitting

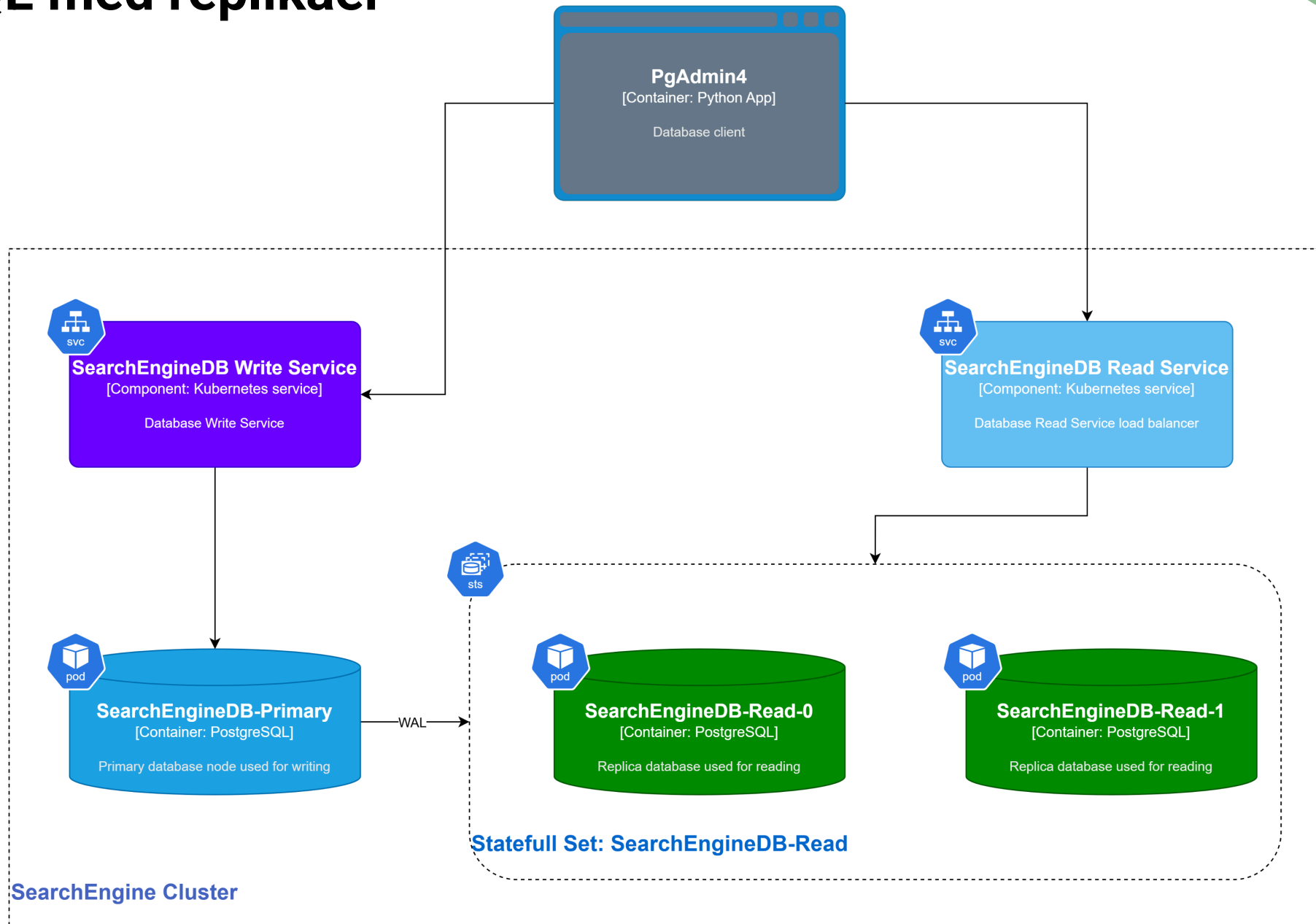
Fordele:

- **Håndterer store datamængder** gennem opdeling.
- **Balanceret belastning** via hashing/modulus.
- **Øget robusthed** med fejlisolering.
- **Bedre performance** via geografisk placering.
- **Fleksibel skalering** af enkelte shards.

Ulemper:

- **Kompleks routing** til rigtige shards.
- **Tunge tvær-shard forespørgsler** kan hæmme performance.
- Kræver **mere drift og overvågning**.
- **Dyr datamigrering** ved rebalancering.
- **Stor planlægningsindsats** ved opstart.

PostgreSQL med replikaer



Opgave M12.01

Canvas opgaver

Postgres med read replicas i K8s

